



Musil

Scripting Language for Sound And Music Computing

User Manual version 0.7.2

Musil is a small scripting language for sound and music computing: Scheme underneath, Tcl on the surface. Numbers are vectors, functions are values, code is data. The language is one C++ header with no dependencies; the libraries add the operating system, linear algebra and machine learning, signal processing, real-time sound and plotting, and the IDE puts all of it in a window.

Contents

1	Overview	2
1.1	Two ways to run Musil	2
1.2	Building and installing	2
1.3	Getting help	2
2	The language	3
2.1	Reading a program	3
2.2	Values	3
2.3	Variables: <code>var</code> and <code>set</code>	4
2.4	Control flow	4
2.5	Functions	4
2.6	Infix arithmetic: <code>expr</code>	5
2.7	Errors	5
2.8	Code as data	5
2.9	Loading files	6
2.10	Sharing and copying	6
3	The IDE	6
4	Programming practices	7
4.1	The shape of a program	7
4.2	Vectors first	7
4.3	Loops and iteration	8
4.4	Records: options, events, rows	8
4.5	Symbols, quoting, code as data	9
4.6	Files, paths and data	9
4.7	Errors, tests, printing	9
4.8	Adding a library	9
4.9	Style at a glance	10
5	Libraries	11
5.1	core	11
5.2	std	14
5.3	system	21
5.4	scientific	23
5.5	signals	29
5.6	live	39
5.7	music	48
5.8	plot	54
6	Extending Musil from C++	56
7	Examples	57

1 Overview

Musil is a language for people who make music with computation: composers, researchers, students. It is deliberately small. The core language fits in one C++ file of under a thousand lines, and every feature in this manual can be reached from that core by loading a library. The design has three commitments:

- **Numbers are vectors.** A scalar is a vector of size one; `(+ v 10)` adds ten to every element; `(sin v)` is elementwise. Signal processing and numerical work are written as arithmetic, and run at C++ speed.
- **Functions are values.** They are created anywhere, stored in lists, passed to `map`, applied to too few arguments to make partial applications, and called in tail position without growing the stack.
- **Code is data.** A program is a list; `'(+ 1 2)` is a list you can build, inspect and `eval`.

The surface is what makes it comfortable: each line is a command, so `print "x ="x` needs no parentheses around it; `( )` is a list; `{ }` is a block of lines; `(expr (a + b * 2))` gives you infix arithmetic when a formula reads better that way.

1.1 Two ways to run Musil

The command line. `musil file.mu` runs a file; `musil` alone is a REPL; `musil a.mu b.mu - x y` runs several files in one interpreter with `args` bound to `(x y)`; `musil -e "print 42"` evaluates a string.

The IDE. *Musil* the application is an editor with syntax colouring where Cmd-Enter runs the block under the cursor, a console with an input line, the list of your variables, a documentation search, and plots and controls in windows of their own. It runs the same interpreter. See section 3.

1.2 Building and installing

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build           # every library has a test and a
                                # reference
cmake --install build           # musil -> /usr/local/bin, *.mu and
                                # help.txt -> ~/.musil
./deploy_macos.sh               # macOS: dist/Musil.app
```

The build fetches FLTK 1.4, the one external dependency, used by the plot library, the controls window and the IDE; `-DMUSIL_FLTK=OFF` builds everything else with no dependency at all. Libraries are found by `load` in `~/.musil` after installing, in any directory listed in the `MUSIL_PATH` environment variable, or next to the file that loads them; during development `MUSIL_PATH=src` points at the source tree.

1.3 Getting help

`(help name)` prints the signature and description of any builtin or library function, taken from the same comments this manual is generated from:

```
> help range
(range n) (range a b) (range a b step)
a vector of evenly spaced numbers, end excluded
```

The files `examples/reference_*.mu` are runnable tours of the core and of each library; `examples/reference.mu` is the best place to start.

2 The language

2.1 Reading a program

A program is a sequence of lines. Each line is a command: its first word is a function and the rest are its arguments.

```
print "hello"           # (print "hello")
var x 10                # (var x 10)
print "x + 1 =" (+ x 1) # a parenthesised form is a
                        sub-expression
```

Inside parentheses, newlines are ordinary whitespace, so a long call can span lines. Braces group lines into a block, which is what `if`, `while` and function bodies take:

```
if (> x 5) {
  print "big"
  var y (* x 2)
}
```

A line holding a single parenthesised form is that form; a line holding a single literal is that value; a line holding a single *symbol* calls it (`ctr` calls the function `ctr`, as in Tcl). A comment runs from `#` to the end of the line; a backslash at the end of a line continues it, and so does **indentation**: at the top level, a line that starts with spaces continues the command above it, which is how a function's body goes on the next line:

```
function lead (gate freq cutoff)
  (pan (* (adsr sr gate 0.01 0.2 0.5 0.4) (lowpass (osc sr freq
    saw-table) sr cutoff 2)) 0)
```

Inside `{ }` lines are separate commands whatever their indentation. `'x` is short for `(quote x)`.

2.2 Values

Type	Literal or constructor	(type x)
number, vector	42, 3.14, (vec 1 2 3), (range 8)	scalar, vec
string	"text\n"	string
symbol	'name, (sym "name")	symbol
list	(list 1 "two" (vec 3 4))	list
function	(function (x) (* x x))	function
nil	nil	nil

All numbers are IEEE double vectors; there is no integer type, and `floor`, `round` and `mod` do integer work. A scalar is a vector of size one, so every arithmetic and math builtin is elementwise, with size-one broadcasting:

```
var v (vec 1 2 3)
(+ v 10)           # (11 12 13)
(* v v)            # (1 4 9)
(sin (/ (range 8) 8)) # elementwise
(+ (vec 1 2) (vec 1 2 3)) # error: +: size mismatch (2 vs 3)
```

Lists hold anything, including other lists and vectors; that is also how matrices (a list of row vectors) and spectra ((list re im)) are represented. Strings are immutable; `getidx` on a string gives a one-character string.

Truth: everything is true except 0, `nil`, an empty string, an empty list, and a vector with a zero in it. Comparisons return 1 or 0, elementwise on vectors; `==` and `equal?` compare structure on non-numbers, and `<` orders two strings lexicographically. Constants: `nil`, `true`, `false`, `pi`, `inf`, `version`.

2.3 Variables: `var` and `set`

`var` defines a variable in the *current* environment: at top level a global, inside a function a local of that call. It never touches a variable of the same name further out, so a library's locals cannot clobber yours. `set` assigns to the nearest existing variable, however far out, and is an error if there is none.

```
var counter 0
function bump () (set counter (+ counter 1))
function shadow () {
  var counter 99          # a different, local counter
  return counter
}
bump
print counter (shadow) counter    # 1 99 1
```

Blocks (`{ }`, loop bodies, `if` branches) do not open a scope: a `var` in a `while` body inside a function is the function's local.

2.4 Control flow

```
if (< x 0) { print "negative" } { print "not negative" }    #
  then-block, optional else-block
while (< i 10) { var i (+ i 1) }
for (var k 0) (< k 10) (var k (+ k 1)) { print k }         # init,
  condition, step, body
if (== k 7) { break }                                     # break and continue are calls, so inside
  a block or in parens
```

There is no `else if`; a chain of `ifs` with early `returns` reads the same way inside a function. `do` evaluates forms in order and returns the last; it is what a block is.

2.5 Functions

```
function square (n) (* n n)                               # named, one-expression body
function fact (n) {                                       # block body; return leaves early
  if (< n 2) { return 1 }
  return (* n (fact (- n 1)))
}
var inc (function (n) (+ n 1))                           # anonymous function in a
  variable
(map (vec 1 2 3) square)                                  # functions are values
```

Closures capture the environment they were created in; a stateful closure updates a captured variable with `set`:

```
function make-counter () {
  var c 0
```

```

    return (function () { set c (+ c 1) })
  }
  var ctr (make-counter)
  ctr                                     # 1
  print (ctr)                           # 2

```

Currying. A function called with fewer arguments than it takes returns a partial application; with more, the result is applied to the rest:

```

function add3 (a b c) (+ a b c)
var add1 (add3 1)
(add1 2 3)                                     # 6
(map (vec 1 2 3) (add3 10 100))               # (111 112 113)
function twice (f) (function (x) (f (f x)))
(twice inc 5)                                 # 7: (twice inc) applied to 5

```

Warning: Because of currying, calling a library function with one argument too few returns a function instead of failing: `(mat-str M)` is a partial application, not a string. Functions have no optional arguments for this reason; where a default makes sense there are two functions (`dist` and `dist-p`, `mat-print` and `mat-print-with`). Builtins do check their arity.

Tail calls run in constant space: a recursive function whose recursive call is the last thing it does can loop a million times. An early `return` inside a block is rewritten to a tail call at definition time, so the natural style costs nothing; only a `return` from inside a loop unwinds.

2.6 Infix arithmetic: `expr`

```

(expr (x + y * 2))                           # precedence as usual: 16 for x
  =10, y=3
(expr ((x + y) * 2))                         # parentheses group
(expr ((sqrt x) + 1))                       # a form without an operator in
  second position is a call
(expr (a > 1 && b < 2))                     # comparisons and logic: < > <= >
  = == != && ||

```

2.7 Errors

```

var r (try (num "abc") catch e (concat "caught: " e))
error "negative not allowed: " n             # raise; the arguments become the
  message
assert (== (+ 2 2) 4) "arithmetic"          # std: stop unless true

```

Every error carries the file, the line and the call stack, innermost first:

```

reference.mu:426: from inner
  1> inner() at reference.mu:428
  2> outer() at reference.mu:431

```

Builtins check the number and types of their arguments (`sqrt`: expected 1 argument, got 0; `+`: expected number, got string). Deep recursion stops with **stack overflow** rather than crashing.

2.8 Code as data

```

var code '(+ 1 2)           # a list, not a call
(eval code)                 # 3
(eval (list '* 2 (+ 1 2)))  # code built at run time: 6
(apply add3 (list 1 2 3))   # a list spread as arguments
(eval (parse "var z (* 6 7)")) # text read into a form, then
    evaluated: a REPL in two calls

```

examples/bootstrap.mu takes this to its end: an evaluator for the language, written in the language in eighty lines (environments, closures, `if`, `var`, `set`, `return`, `while`), with a prompt at which every line is read by the host and evaluated by it.

2.9 Loading files

(load "file.mu") runs a file once in the global environment. A relative name is searched next to the loading file, in the current directory, in each entry of `MUSIL_PATH`, then in `~/musil`. Reading functions (`read`, `read-wav`, `read-csv`, `exists?`, `stat`) apply a similar rule to their data: a relative path that does not exist in the current directory is looked for next to the file being run, so a script finds its data from wherever it was started, on the command line or from the IDE. A file that ran is not run again (so mutual loads are safe); a file that failed is not cached. Nothing is loaded automatically: a program that wants the standard library says `load "std.mu"`; each library loads what it needs (`signals.mu` loads `scientific.mu`, which loads `std.mu`).

2.10 Sharing and copying

Lists and vectors are shared by reference: (`var alias orig`) makes two names for one object, and `push` or `setidx` through either is visible through both. (`copy x`) makes a new list or vector with the same elements (a shallow copy). Strings and numbers behave as values.

3 The IDE

Musil the application is the interpreter with an editor around it.

- **Editor** (top left): .mu files with syntax colouring (keywords, the documented builtins, strings, numbers, comments). **Cmd-Enter** runs the block around the cursor (the lines between blank lines) or the selection; **Cmd-Shift-Enter** the whole file; **Cmd-Alt-Enter** the current line. The lines sent flash. A toolbar has Run file, Run block, Run line, Stop, Clear, Controls and Settings. Undo, find (Cmd-F, Cmd-G), comment/uncomment (Cmd-/), text size (Cmd+, Cmd-, for every panel), go to line (Cmd-L), clear the console (Cmd-K), files by drag and drop; Settings (Cmd-,) chooses the font (one fixed-width font for everything, colours only mark syntax), the size, line numbers, the current-line highlight, the margin guide and the evaluation port (0 turns it off), and remembers them.
- **Console** (bottom left): everything printed, and a line to type Musil into: Enter runs it, Up/Down recall history, Tab completes a name. Esc or Cmd-. stops a running program (the interpreter checks at safe points, so the session survives).
- **Variables** (top right): the globals with a preview of their values, coloured by kind (numbers, strings, lists, functions); double-click prints one.
- **Help** (bottom right): type a name or a word to search the documentation of every function. (manual) or the Help menu opens this PDF; Help → Examples copies any bundled example into an untitled buffer (the original stays untouched; its data is still found), to run and edit.

- **Plots and controls** open in windows of their own, from the IDE exactly as from the command line; several plots can be open at once.
- **From another editor:** the IDE listens on port 7770; code sent from VS Code (the extension in `editors/vscode/musil`) or from `nc` is evaluated in the same session.

The interpreter runs on its own thread; the window stays responsive while a program runs. Libraries are found inside the app bundle, so nothing needs to be installed. On macOS, `./deploy_macos.sh` builds `dist/Musil.app`.

4 Programming practices

How to write Musil that reads well and runs fast, and how to extend it. The language is a Scheme in its bones (everything is an expression, functions are values, code is data) with a Tcl surface (one command per line, no parentheses around a top-level call). The style that fits it best is *imperative at the top, functional in the small*: programs are sequences of steps that build and transform vectors and lists; the functional tools (`map`, `filter`, `reduce`) are used inside those steps where they are shorter than a loop, never as an end in themselves.

4.1 The shape of a program

A program is a list of steps. Name the intermediate results; do not nest what can be sequenced.

```
load "signals.mu"
var w (read-wav "data/drums.wav")
var sr (head w)
var x (head (getidx w 1))
var flux (onset-strength x 1024 256)
var t-on (onsets x sr 1024 256 0.25)
print (length t-on) "onsets:" (fixed t-on 3)
write-wav "/tmp/out.wav" sr (normalize-peak x)
```

- One idea per line. A nesting deeper than three parentheses is asking to be split with a `var`.
- Top-level calls need no parentheses: `print x`, `write-wav path sr x`, `audio-init`. Parentheses are for calls that are arguments: `var y (fixed x 2)`.
- A function's body goes on the next line, indented: indentation continues a top-level command. Braces for several statements. No backslashes are needed.
- `return` ends a function early; the last expression is the value otherwise. Inside `{ }`, use `return` when the body has steps, so the reader sees where the value comes from.
- `and` and `or` short-circuit, left to right, and return the deciding value, so `(and (> k 0) (getidx v k))` is safe and `(or x default)` reads as it should.

```
function rms-db (x)
  (db (rms x))

function normalise-to (x target) {
  var g (/ target (rms x))
  return (* x g)
}
```

4.2 Vectors first

Numbers are vectors. Anything that can be written as an operation on whole vectors is faster (the loop runs in C++) and shorter than a loop in Musil.

```

var t (/ (range n) sr)                                # a time axis
var y (* (exp (* -3 t)) (sin (* tau 440 t)))           # a decaying sine,
    no loop
var peak (max (abs y))
var clipped (clip y -0.5 0.5)

```

Broadcasting: a scalar with a vector applies to every element; two vectors must have the same length (a mismatch is an error, on purpose). `vec` concatenates; `slice`, `take`, `drop` cut; `add-at!` adds into a buffer in place; `mix` builds a buffer from placed pieces. Reach for a loop over samples only when each sample depends on the previous one and no builtin does it (`osc`, `lag`, `iir`, `delay`, `comb`, `adsr` are there); such a loop usually belongs in C++ (section 4.8).

4.3 Loops and iteration

Three forms, in order of preference. `each` for doing something with every element (printing, pushing, writing files): the everyday loop. Over a vector it gives numbers, over a list its elements; `(range n)` is the counter loop. Inside the function, `set` reaches variables of the enclosing scope.

```

each files (function (f) (print f (fixed (rms (head (getidx (read-wav
    f) 1))) 3)))
each (zip names sources) (function (p) (write-wav (concat "/tmp/" (
    head p) ".wav") sr (last p)))
each (range 8) (function (k) (push hits (list (* k 4000) burst)))

```

`for` when an index and a step are the point, or the loop must stop early; `while` for conditions that are not a count (reading until a value, converging, waiting). `break` and `continue` work in both.

```

for (var k 1) (< k (length mags)) (var k (+ k 1)) {
    push flux (spectral-flux (getidx mags k) (getidx mags (- k 1)))
}
while (controls-open?) { sleep 0.1 }

```

`map`, `filter`, `reduce` when the result is a new collection: `map` turns a list into another of the same length, `filter` keeps some elements, `reduce` folds into one value. Use them when the function fits on the line; otherwise name the function first. `map` over a list gives a list; over a vector it gives a vector when every result is a number and a list otherwise, so `(map (range n) (function (k) (list k ...)))` builds a list of records directly.

```

var levels (map sources rms)
var loud (filter events (function (e) (> (get e 'amp) 0.5)))
var total (reduce lengths + 0)

```

Recursion is for trees and for algorithms that are recursive by nature (a nested pattern, an evaluator, a parser). The interpreter turns a call in tail position into a jump, so a loop written as a tail-recursive function costs nothing, but it reads worse than `for` to most people; prefer the loop unless the structure is genuinely recursive. The evaluation depth is bounded (`-stack`), so a non-tail recursion over a long list will overflow: use a loop or `reduce`.

Building lists: start with `(list)` and `push`, or `map`. Do not `concat-list` in a loop over long lists (each call copies). For vectors, preallocate with `zeros` and write with `add-at!` or `setidx`.

4.4 Records: options, events, rows

A record is a list of `(list key value)` pairs. It is the shape of every option list, every event, every row of a table, so its helpers are used everywhere: `record`, `get`, `opt` (with a default), `has?`,

`put!` (in place) and `put` (a copy), `keys`, `values`; and, over lists of them, `sort-by`, `group-by`, `find-first`, `reject`, `min-by`, `max-by`, `sum-by`, `partition`. Keys are symbols by convention (`'freq`); a string key is a different key, so pick one and keep it.

```
var note (record (list 'pitch 60 'dur 0.5 'amp 0.8))
put! note 'amp 0.6
var louder (filter notes (function (n) (> (get n 'amp) 0.5)))
var by-pitch (sort-by notes (function (n) (get n 'pitch)))
```

4.5 Symbols, quoting, code as data

`'x` is the symbol `x`: a name as a value, used for keys, kinds and parameter names (`(set-param a 'cutoff 300)`). Quote a whole form to keep it as data: `'(+ 1 2)` is a list of three elements. `eval` runs a form, `parse` turns text into forms, `apply` calls a function on a list of arguments. This is what makes patterns, scores and instruments plain data you can build and transform; it is rarely needed in ordinary programs, and never as a way to generate code when a function will do.

4.6 Files, paths and data

- Reading: `read-wav`, `read-csv`, `read`, `read-lines`. A relative path is looked for in the current directory, then next to the file being run, so `data/drums.wav` works from anywhere for an example living in `examples/`. `load` and `find-file` also search the load path (`MUSIL_PATH`, `~/musil`, the libraries).
- Writing: `write-wav`, `write-csv`, `write`. Relative paths write into the current directory; the examples write into `/tmp/` on purpose, so that running them never litters the tree; outputs are prefixed `musil_`.
- WAVs come back as `(list sr channels)`; a mono file is one channel: `(head (getidx w 1))`. A multichannel signal is a list of channel vectors, which `write-wav` and `play` take as they are.
- Big files: slice what you need early, keep sample rates in a variable, never hard-code 44100 in a function: pass `sr`.
- Data the examples share lives in `examples/data/` and is committed; how each synthetic file was made is on record in `make_audio_demos.mu`. Data a library needs (the HRTFs) lives in `src/` and is installed with the `.mu` files.

4.7 Errors, tests, printing

`error` raises with a message; `try ... catch e` catches it; `error-of` gives the message of a failing thunk, which is how tests check errors. Check arguments at the top of a function and fail early with a message that names the function: `if (<= n 0){ error "hann: n must be > 0" }`. `print` takes any number of arguments; `fixed` rounds for display; `str` converts. A program should be silent when it succeeds and precise when it fails.

4.8 Adding a library

A library is four files plus registration; `signals` is the model.

1. `src/name.h`: the C++ half, for anything needing an element loop, per-sample state, the OS, or a callback. One inline `vp_ptr sig_thing(vlist& a, Interp& i)` per builtin, arguments checked with `i.num`, `i.scalar`, `i.str`, `i.list`, `i.index`, errors with `i.bad("thing: what was wrong")`; then inline `void add_name(Interp& i)` registering them with `i.def("thing", sig_thing, min, max)`. Everything is inline in namespace `musil` (header-only).

2. `src/name.mu`: the Musil half, for anything expressible as vector operations. It starts with `load "std.mu"` (and whatever it builds on). Loading is explicit: a program does `load "name.mu"`.
3. `tests/test_name.mu`: self-checking. `load "test.mu"`, then `check` lines with a message each, then `report "test_name"`. Cover the normal case, an edge (empty, one element), and the errors. Register it in `CMakeLists.txt`.
4. `examples/reference_name.mu`: a tour that prints every function once, in the order a reader would want to meet them; its output is a golden file in `tests/golden/`, refreshed with `tests/golden.cmake -DUPDATE=1` whenever the printed text changes on purpose.

Then `#include "name.h"` and `add_name(i)` in `src/musil.h`; the library in LIBS of `tools/gendoc.py`; `\input{generated/name.tex}` and a paragraph in this manual.

Rules that keep the libraries coherent:

- **Doc comments are the documentation.** `//` (thing args) what it does above a builtin in the `.h`, `#` (thing args) what it does above a function in the `.mu`; several forms on one line share the text; indented continuation lines extend it. `help`, this manual and the VS Code grammar are generated from these; a function without one is a bug.
- **No optional arguments.** Currying makes a missing argument a partial application, so a function has one arity; variants get names (`onsets` / `onsets-adaptive`).
- **Names** are lowercase with hyphens; predicates end in `?`; in-place mutation ends in `!` (`add-at!`, `put!`); a function returning a new value has no mark.
- **Element loops in C++, vector algebra in Musil.** A Musil function that loops over samples is a sign the loop belongs in the `.h`.
- **Offline and streaming are one definition.** A signal function that should also run in a synth takes a state struct in C++ (`osc_run`, `iir_run` ...), is listed in the ugen table in `live.h`, and must satisfy `synth-render` equals the direct call (a test in `test_live.mu`). Constants inside instruments are signals (`(sig gate 190)`), so they work in both executions.
- **Environment variables the tests rely on:** `MUSIL_NOSHOW` (no windows), `MUSIL_NULL_AUDIO` (the silent device). A library that opens windows or devices honours them.
- **The version** is set once, in `src/core.h`; CMake, the deploy scripts, the VS Code extension and this manual read it from there.

Adding a builtin to an existing library is the same, smaller: the function and its doc comment, a `check` in the test, a line in the reference, a regenerated golden; `./build.sh -test` before sending.

4.9 Style at a glance

```
# a comment says why, the code says what
load "live.mu"
audio-init
var sr (audio-sr)

function pluck-note (gate freq)                                # instrument: a
    function of its parameters
    (* (adsr sr gate 0.005 0.1 0.3 0.2) (osc sr (sig gate freq)
        saw-table))

var p (synth pluck-note)                                        # streamed
var buffer (pluck-note (vec (ones 400) (zeros 8000)) 220)      # or
    rendered: the same function
```

```
function bassline (cycle)                                # a pattern: events
  from data
  (melody p "a1 ~ a1 c2" 4 (list))
live-loop 'bassline 4
```

Prefer this to cleverness: named steps, vectors over loops, **each** over recursion, records for structured data, one arity per function, a doc line on every definition.

5 Libraries

A library is a pair of files: **name.h** holds the functions that need C++, an element-by-element loop or the operating system; **name.mu** holds everything that is a composition of vector operations, which runs at the same speed in Musil. The C++ halves are registered when the interpreter starts; the Musil half is loaded with `load "name.mu"`. Each library has a test (`tests/test_name.mu`) and a runnable reference (`examples/reference_name.mu`).

Library	Load	Contents
core	built in	the language, arithmetic, lists, vectors, strings, errors, load
std	<code>load "std.mu"</code>	vector constructors and statistics, sequences, strings, files, map/filter/reduce
system	<code>load "system.mu"</code>	processes, directories, CSV, WAV, UDP/OSC
scientific	<code>load "scientific.mu"</code>	matrices, decompositions, regression, PCA, k-means, KNN
signals	<code>load "signals.mu"</code>	generators, FFT, STFT, phase vocoder, features, filters, reverb
live	<code>load "live.mu"</code>	the audio device, a sample-accurate clock, voices playing buffers and files
plot	<code>load "plot.mu"</code>	figures of lines, points, bars, images and surfaces; windows and PNGs

The function references that follow are generated from the doc comment above each function in the source, the same text that `help` prints.

5.1 core

Always available, no `load` needed. Special forms are listed first.

(quote x)

'x: x itself, unevaluated: code as data

(do forms...)

evaluate in order, return the last; what { } builds

(if cond then [else])

cond is true unless it is 0, nil, empty, or a vector with a zero

(while cond body)

loop; (break) and (continue) inside it

(for init cond step body)

C-style loop: (for (var i 0) (< i 10) (var i (+ i 1)) { ... })

(var name value)

define name in the current environment; a function's var is always local

`(set name value)`

assign to the nearest existing name, however far out; error if none

`(function name (params)`

body) define a function; (function (params) body) an anonymous one

`(return [value])`

leave the function; free in tail position

`(break) (continue)`

leave or restart the innermost loop

`(try body catch e handler)`

run body; on an error bind its message to e and run handler

`(expr (a op b ...))`

) infix arithmetic: + - * / % < > <= >= == != && ||; () groups, (f x) calls

`(parse "text")`

read Musil source into a form: (do line ...), the same reading a file gets; (eval (parse s)) runs it

`(eval form)`

evaluate a form (code built as data) in the global environment

`(apply f list)`

call f with the elements of list as arguments

`(+ a b ...) (- a [b ...]) (* a b ...) (/ a [b ...])`

arithmetic, elementwise with size-1 broadcasting; (- x) negates, (/ x) is the reciprocal, (+) is 0 and (*) is 1

`(< a b) (> a b) (<= a b) (>= a b)`

comparisons: elementwise 1/0 on numbers, lexicographic on two strings

`(== a b) (!= a b)`

equality: elementwise on numbers, structural on anything else

`(equal? a b)`

structural equality as a single 1/0, also for vectors and nested lists

`(sin x) (cos x) (tan x) (asin x) (acos x) (atan x) (sqrt x) (exp x) (log x)`
`(log2 x) (log10 x)`

trigonometry and logs, elementwise

`(abs x) (floor x) (ceil x) (round x)`

rounding and magnitude, elementwise

`(tanh x)`

hyperbolic tangent, elementwise (a soft clip for signals)

`(mod a b)` `(pow a b)` `(atan2 y x)`

remainder (sign follows fmod), power, two-argument arctangent; elementwise

`(not x)`

the negation of a truth value

`(and a b ...)` `(or a b ...)`

short-circuit, left to right: `(and)` stops at the first false value, `(or)` at the first true one; the last argument is returned as it is, so `(or x default)` is `x` when `x` is true. Special forms: later arguments are not evaluated when the answer is known, so `(and (> k 0) (getidx v k))` is safe.

`(min v)` `(max v)`

smallest or largest element of one vector; with several arguments, elementwise over them

`(list x ...)`

a list of anything

`(append L x ...)`

a new list with `x ...` at the end

`(push L x)`

append in place, returning the list; `(pop L)` remove and return the last element

`(append L x ...)`

a new list with `x ...` added at the end

`(pop L)`

remove and return the last element of a list, in place

`(length x)` `(head x)` `(tail x)` `(empty? x)`

on a list, a vector or a string

`(getidx x k)`

element `k` of a list, vector or string; negative `k` counts from the end

`(setidx x k v)`

set element `k` of a list or vector in place

`(vec x ...)`

a vector from numbers, vectors and lists of numbers, concatenated; a scalar is a vector of size 1

`(sum v)`

sum of the elements (0 for an empty vector)

`(print x ...)`

print the arguments separated by spaces, then a newline

`(exit [code])`

end the program

`(type x)`

"scalar" "vec" "string" "symbol" "list" "function" "nil" or "opaque:tag"

`(str x)`

x printed as a string

`(sym s)`

a symbol from a string; (num s) a number from a string

`(copy x)`

a shallow copy: a new list or vector with the same elements

`(error x ...)`

raise an error with the arguments as message

`(clock)`

seconds from a monotonic clock, for timing

`(load "file.mu")`

run a file once, searched next to the loading file, in the current directory, in MUSIL_PATH and in ~/.musil

`(defined? 'name)`

is name bound?

`(vars)`

the global names, sorted

`(help name)`

print the documentation of a builtin or library function (from help.txt)

`(num s)`

the number written in a string (or a number unchanged)

5.2 std

The general-purpose standard library. Vectors: **range** is the constructor everything else is built on (**zeros**, **ones**, **linspace**), then statistics (**mean**, **stdev**, **median**, **quantile**, **histogram**), and **interp1** for table lookup. Sequences: **slice**, **take**, **drop**, **find**, **zip**, **unique**. Strings: **split**,

join, format, fmt-fixed. Files: read, write. Higher-order: map, filter, reduce, each, all of them on lists and vectors alike.

```
load "std.mu"
var v (range 0 10 0.5)
print (mean v) (stdev v) (quantile v 0.9)
print (map (list "a" "b") upper) (filter v (function (x) (> x 7)))
print (format "{} of {}" (count v even?) (length v))
```

(range n) (range a b) (range a b step)

a vector of evenly spaced numbers, end excluded

(seed n)

seed the random generator so that rand repeats

(rand)

a number in [0, 1); (rand n) a vector of n of them

(sort v)

the elements of a vector in ascending order

(reverse x)

a list, vector or string reversed

(slice x start [count])

part of a list, vector or string; negative start counts from the end

(find x v)

index of the first element equal to v, or of the substring v in a string; -1 if absent

(concat x ...)

the arguments printed and joined into one string

(split s sep)

the pieces of s between occurrences of sep; (split s "") the characters

(join L sep)

the elements of L printed and joined with sep

(format "text {} more {}" x y)

each {} replaced by the next argument

(upper s) (lower s) (trim s)

case changes and whitespace trimming

(chr n)

the character with code n

`(ord s)`

the code of the first character of `s`

`(read path)`

the whole file as a string; a relative path that does not exist in the current directory is looked for next to the file being run (so a script finds its data from anywhere)

`(write path x)`

write `x` (printed) to a file, replacing it

`(append-file path x)`

write `x` (printed) at the end of a file

`(directory? path)`

is the path an existing directory?

`(exists? path)`

does the file or directory exist?

`(input [prompt])`

a line read from the console, or nil at end of input

`(map x f)`

`f` applied to every element: a list for a list; for a vector, a vector when every result is a number, a list otherwise

`(filter x f)`

the elements for which `f` is true, as a list or vector like `x`

`(reduce x f init)`

fold: `(f (f (f init x0) x1) x2) ...`

`(assert cond [message ...])`

stop with an error unless `cond` is true

`(regex-match? pattern s)`

does the regular expression (ECMAScript syntax) match anywhere in `s`?

`(sign x)`

-1, 0 or 1

`(even? n) (odd? n)`

`(between? x lo hi)`

`lo <= x <= hi`

(hypot a b)

$\sqrt{a^2 + b^2}$

(gcd a b) (lcm a b)

on non-negative integers

(lerp a b t)

linear interpolation, t in $[0, 1]$

(map-range x in-lo in-hi out-lo out-hi)

re-map x from one range into another

(deg->rad d) (rad->deg r)

(zeros n)

vector of n zeros

(ones n)

vector of n ones

(linspace a b n)

n points from a to b inclusive

(mean v)

arithmetic mean

(prod v)

product of the elements

(dot a b)

inner product (sizes must match)

(norm v)

euclidean norm

(cumsum v)

running sum

(stdev v)

population standard deviation

(normalize v)

scale to $[0, 1]$; a constant vector becomes zeros

(diff v)

first differences, one element shorter

`(argmin v) (argmax v)`

index of the smallest / largest element

`(median v)`

the middle value (mean of the two middle values for even length)

`(quantile v q)`

value below which a fraction q of the data lies (linear interpolation)

`(histogram v bins)`

counts per bin over [min v, max v]; returns (list counts edges)

`(interp1 xs ys xq)`

linear interpolation of the table (xs, ys) at the points xq; xs must be increasing; xq outside the table is clamped to the ends

`(clip v lo hi)`

clamp every element to [lo, hi]

`(fixed x d)`

round x to d decimal places

`(last x)`

last element

`(take x n)`

first n elements

`(drop x n)`

all but the first n elements

`(vec->list v)`

the elements of a vector as a list

`(contains? x v)`

is v an element of x (or a substring, for strings)?

`(each x f)`

call f on every element, for side effects; returns nil

`(any? x f)`

does f hold for some element?

`(all? x f)`

does f hold for every element?

`(count x f)`

number of elements for which f holds

`(concat-list a b)`

a new list with the elements of a then those of b

`(flatten L)`

one level of nested lists removed

`(zip a b)`

list of two-element lists

`(unique L)`

elements in first-seen order, duplicates removed

`(sort-list L)`

sort a list of numbers or strings (insertion sort; fine for short lists)

`(min-of L) (max-of L)`

smallest / largest element of a list of numbers

`(repeat s n)`

string s repeated n times

`(starts-with? s prefix) (ends-with? s suffix)`

`(replace s old new)`

every occurrence of old replaced by new

`(count-substr s pat)`

number of non-overlapping occurrences of pat

`(pad-left s w ch) (pad-right s w ch)`

pad to width w with character ch

`(fmt-fixed x d)`

x as a string with exactly d decimals: `(fmt-fixed pi 4)` => "3.1416"

`(fmt-pct x d)`

x as a percentage string: `(fmt-pct 0.752 1)` => "75.2%"

`(opt rec key default)`

the value of key in a record, or default; keys compare with equal? (symbols and strings alike)

`(get rec key)`

the value of key; an error when absent

`(has? rec key)`

is the key present?

`(put! rec key value)`

set a key in place (replacing an existing pair, or adding one); returns the record

`(put rec key value)`

a new record with the key set, the original untouched

`(keys rec) (values rec)`

the keys, the values, in order

`(record key value ...)`

a record from alternating keys and values: `(record 'freq 440 'dur 0.5)`

`(sort-by L f)`

the elements ordered by f (a number or string per element), stable

`(group-by L f)`

a record from key to the list of elements with that key, in first-seen order

`(find-first L f)`

the first element for which f holds, or nil

`(reject L f)`

the elements for which f does not hold

`(min-by L f) (max-by L f)`

the element with the smallest / largest f

`(sum-by L f)`

the sum of f over the elements

`(partition L f)`

(list those-for-which-f-holds the-others)

`(remove-at L k) (insert-at L k x)`

a new list without / with an element at k

`(identity x)`

`(compose f g)`

the function `x -> (f (g x))`

`(times n f)`

call `f` `n` times with `0 .. n-1`

`(assert-near a b eps msg)`

numeric comparison within `eps`

`(assert-equal actual expected msg)`

structural comparison with a helpful message

5.3 system

Access to the operating system. `read-wav` returns `(list sample-rate (list channel ...))`, each channel a vector, which is the representation every signal function uses; `write-wav` takes a vector (mono) or a list of vectors. CSV tables are lists of rows, numeric cells become numbers.

```
load "system.mu"
var w (read-wav "sound.wav")
var sr (head w)
var left (head (getidx w 1))
write-wav "/tmp/out.wav" sr (* 0.5 left)
print (file-lines "notes.txt") (ls ".") (getenv "HOME")
```

`(exec "command")`

=> the command's standard output as a string

`(getenv "NAME")`

=> value or nil

`(sleep seconds)`

`(find-file "name")`

=> the path of a file as load resolves it (the current directory, the file being run, the load path: `MUSIL_PATH`, `~/musil`, the libraries), or "" when not found

`(resolve-path "name")`

=> the path a reading function would open for a relative name: as given if it exists from the current directory, else next to the file being run, else next to the program that was started

`(interactive?)`

=> is the standard input a terminal? (a script asks before prompting for input)

`(now)`

=> seconds since the Unix epoch (wall clock; clock is the monotonic one)

`(cwd)`

=> current directory

`(ls [dir])`

=> list of entry names, sorted

`(mkdir path)`

=> creates the directory and its parents; nil

`(remove path)`

=> deletes a file or an empty directory; 1 if something was removed

`(stat path)`

=> (list size is-dir modified-seconds) or nil if it does not exist

`(read-csv path)`

=> table. write-csv is in system.mu.

`(read-wav path)`

=> (list sample-rate (list channel-vector ...)); 16-bit PCM or 32-bit float WAV

`(write-wav path sample-rate channels [bits])`

=> nil. channels: a vector (mono) or a list of vectors; bits 16 (default) or 32 (float)

`(udp-send host port payload [osc?])`

=> 1 on success, 0 on failure. With osc?=1 the payload is

`(udp-receive host port [timeout-seconds])`

=> the next datagram as a string, or nil on timeout/failure

`(serve port)`

start the evaluation port; (serve-stop); (serving) => the port or 0

`(file-lines path)`

list of lines, without the trailing newline

`(write-lines path lines)`

write a list of lines, one per line

`(file? path) (dir? path)`

existence tests by kind

`(file-size path)`

size in bytes, or nil

`(basename path) (dirname path) (extension path)`

string surgery on paths

`(csv-escape s)`

quote a field if it holds a comma, a quote or a newline

`(write-csv path table)`

table is a list of rows, a row a list of cells

`(elapsed thunk)`

seconds taken by calling thunk

`(wav-duration w)`

seconds of a value returned by read-wav

5.4 scientific

Linear algebra, statistics and machine learning. A matrix is a list of row vectors: `(list (vec 1 2) (vec 3 4))` is 2×2 ; `(getidx M i)` is a row. Products, transposition, elimination (`det`, `inv`, `solve`), the symmetric eigensolver, k-means and KNN are C++; construction, elementwise arithmetic, statistics along an axis, covariance, PCA, regression, NMF and the model helpers are Musil.

```
load "scientific.mu"
var A (list (vec 2 1) (vec 1 3))
print (mat-mul A (inv A)) (solve A (vec 3 5)) (eig-sym A)
var P (pca X)                                # rows: direction then
      eigenvalue, largest first
var km (kmeans X 3)                          # (list labels centroids)
var model (knn-model training 3)
print (accuracy (knn-test model test) test)
```

`(mat-mul A B ...)`

matrix product, left to right

`(transpose M)`

`(det M)`

`(rank M)`

`(inv M)`

by Gauss-Jordan on $[M \mid I]$

`(solve A b)`

$\Rightarrow x$ with $A x = b$

`(median-filter v order)`

the running median of width order

`(eig-sym M)`

=> (list eigenvalues eigenvectors) of a symmetric matrix by Jacobi rotations;

`(kmeans M k)`

=> (list labels centroids): labels is a vector of cluster indices, centroids a k x d matrix

`(knn training k queries)`

=> list of predicted labels.

`(list->mat L)`

a matrix from a list of lists of numbers

`(mat-fill r c v)`

r x c matrix filled with v

`(mat-zeros r c) (mat-ones r c)`

matrices of zeros or ones

`(mat-ones r c)`

a matrix of ones

`(mat-rand r c)`

r x c matrix of values in [-1, 1]

`(eye n)`

identity; (diag v) diagonal matrix from a vector

`(diag v)`

a square matrix with v on the diagonal

`(mat-diag M)`

the diagonal of a matrix as a vector

`(nrows M) (ncols M)`

the number of rows or columns

`(ncols M)`

the number of columns

`(mat-shape M)`

(list rows cols)

`(mat-get M i j)`

element; (mat-set M i j v) in place

`(mat-set M i j v)`

set an element in place

`(mat-row M i)`

row *i* as a vector; `(mat-col M j)` column *j* as a vector

`(mat-col M j)`

column *j* as a vector

`(get-rows M from n)`

n rows starting at *from*, as a matrix

`(get-cols M from n)`

n columns starting at *from*, as a matrix

`(mat-vec A x)`

A *x* as a vector; `(vec-mat x A)` *x* *A* as a vector

`(vec-mat x A)`

x *A* as a vector

`(outer u v)`

the matrix $u v'$, rows $u[i] * v$

`(trace M)`

sum of the diagonal

`(mat-add A B)` `(mat-sub A B)` `(hadamard A B)`

elementwise sum, difference and product of two matrices

`(mat-sub A B)`

elementwise difference

`(hadamard A B)`

elementwise product

`(mat-scale M s)` `(mat-shift M s)`

every element times *s*, or plus *s*

`(mat-shift M s)`

every element plus *s*

`(mat-map M f)`

f applied to every element

`(mat-round M d)`

every element rounded to *d* decimals (for printing)

`(vstack A B)`

rows of A then rows of B

`(hstack A B)`

each row of A followed by the row of B

`(col-stack vectors)`

a matrix whose columns are the given vectors: `(col-stack (list x y))`

`(lp-norm v p)`

$(\sum |v|^p)^{1/p}$; $p = 2$ is the euclidean norm

`(dist x y)`

euclidean distance; `(dist-p x y p)` with another p

`(dist-p x y p)`

distance with another p : 1 for the city-block distance

`(dist-matrix M)`

pairwise euclidean distances between the rows of M

`(nearest M x)`

index of the row of M closest to x

`(mat-sum M axis)`

`(mat-mean M axis)`

`(mat-std M axis)`

population standard deviation

`(zscore M)`

columns standardised to mean 0, std 1 (constant columns become 0)

`(standardize M)`

the same as zscore

`(center M)`

each column minus its mean

`(cov M)`

sample covariance of the columns (rows are observations)

`(corr M)`

correlation of the columns; constant columns give 0

`(linefit x y)`

(slope intercept) of the least-squares line through the points

`(mat-str M d)`

one line per row, d decimals, columns aligned

`(mat-print M)`

print with 3 decimals; `(mat-print-with M d)` chooses the decimals

`(mat-print-with M d)`

print with d decimals

`(linreg-fit X y)`

coefficients b with $y \sim X b$, by the normal equations; y is a vector

`(linreg-predict X b)`

X b as a vector

`(linreg-residuals X y b)`

`(add-intercept X)`

a column of ones in front of X, for a model with a constant term

`(pca X)`

=> d x (d+1) matrix: row k is the k-th principal direction followed by its eigenvalue, largest first

`(pca-directions P)`

the directions of a pca result as a d x d matrix

`(pca-eigenvalues P)`

its eigenvalues as a vector

`(pca-scores X k)`

X projected on the first k principal directions, n x k

`(pca-explained P)`

fraction of variance per component

`(kmeans-labels result) (kmeans-centroids result)`

the two parts of a kmeans result

`(kmeans-centroids result)`

the centroids of a kmeans result, $k \times d$

(cluster-sizes labels k)

how many points fell in each of the k clusters

(knn-model training k)

a model: (list training k); training is a list of (list features label)

(knn-predict model queries)

labels for a list of feature vectors

(knn-predict-one model q)

(knn-test model samples)

predictions for samples given as (list features label)

(accuracy predictions samples)

fraction of predictions equal to the samples' labels

(train-test-split samples fraction)

(list training test), first part of the list

(shuffle L)

a new list in random order (Fisher-Yates)

(nmf V k iterations)

V ($r \times c$, non-negative) $\sim W H$ with W $r \times k$ and H $k \times c$, both non-negative: Lee & Seung's multiplicative updates for the KL divergence (the usual choice for magnitude spectrograms); $\Rightarrow$ (list $W H$). Columns of W are the parts (spectral shapes), rows of H their activations over time.

(nmf-with-parts V W iterations)

the activations H of known parts W (their columns), $V \sim W H$ with W fixed: supervised NMF, when the parts were learnt from examples of each source

(mat-min M) (mat-max M)

the smallest and the largest element

(mat-div A B)

elementwise quotient

(nmf-error V W H)

the KL divergence between V and $W H$ (what the updates decrease)

5.5 signals

Offline signal processing on vectors. A signal is a vector of samples; a complex spectrum is (list re im); a polar spectrum is (list mag phase). The FFT, the table oscillator, the IIR filter, delay, resampling, autocorrelation, peak picking and gather are C++; generators, windows, convolution, the STFT and its inverse, cepstral envelopes, the phase vocoder, the spectral and temporal descriptors, the RBJ biquads, comb and allpass sections and the Schroeder reverb are Musil, as compositions of vector operations. **hpss** (harmonic/percussive separation by median filters over the spectrogram) is C++; **onset-strength**, **onsets** (a global threshold), **onsets-adaptive** (a threshold on the flux's moving median, for material whose loudness drifts) and **segments** build onset detection from the spectral flux in Musil; onsets always start with 0 and are placed at the sample where the energy rises.

Separation by NMF. **nmf V k iterations** (in scientific, fifteen lines of Musil on the C++ matrix product: Lee and Seung's multiplicative updates for the KL divergence) factors a non-negative matrix into k parts; **nmf-separate x n hop k iterations** applies it to the magnitude spectrogram and splits the sound into k sources by Wiener masks that add up to the input. Supervised separation, when examples of each source exist: **nmf-learn-parts** learns a few parts from each example alone, **nmf-separate-with** learns only the activations on the mix (**nmf-with-parts**) and returns one source per instrument; **examples/nmf.mu** does both on a four-stem mix (drums, bass, chords, melody, with each stem alone to score against; made, like every synthetic demo sound, by **make_audio_demos.mu**).

Spatial. Conventions as in AmbiX: azimuth in degrees, 0 in front, positive to the left; elevation up; a multichannel signal is a list of channel vectors, which **write-wav** and **play** take. **pan-azimuth** is stereo from a direction; **pan-n** feeds a ring of speakers (amplitude panning between the two nearest); **ambi-encode** places a mono signal as B-format ($((\text{order}+1)^2)$ channels, ACN/SN3D, orders 0–7) with **ambi-gains** the spherical harmonics behind it; **ambi-rotate** turns a whole field exactly; **ambi-decode** samples the field at a speaker layout (max-rE weighted); **ambi-render** places several sources, **moving-source** one that moves. For headphones, **hrir** gives the two ears' impulse responses of a direction from the measured MIT KEMAR set (710 directions, full sphere, 128 taps; shipped as **hrtf_kemar.csv** with the libraries, loaded on first use, resampled to the session's rate) or, with **hrtf-unload** or when the file is absent, from a spherical-head model (**hrir-model**: interaural delay and head shadow only, so left and right but no front/back or height, which are cues of the pinna that only measured responses carry); **hrtf-load** takes another set in the same CSV layout. **binaural** renders a source through it, **binaural-decode** renders B-format through ten virtual speakers. Every one of these streams in a synth as well (**live**); **binaural** as a node crossfades its responses when the direction moves, so a sound can be flown around the listener from two sliders. **examples/spatial.mu**, **flyby.mu** (a helicopter in front, overhead, behind and around, also as third-order B-format for an external decoder such as the IEM BinauralDecoder in a DAW), **flyby_controls.mu** and **live_spatial.mu** do all of it.

```
load "signals.mu"
var x (sine 44100 440 1)
var frames (stft x 2048 512)
var y (istft frames 2048 512)           # exact reconstruction
var slow (pvoc-stretch x 2)           # twice as long, same
    pitch
var up (pvoc-pitch-formant x 1.5 400)  # a fifth up, formants
    kept
var any (pvoc x (list (list "stretch" (list 1 3)) (list "pitch" 0.75)
    (list "envelope" 400)))
var lp (lowpass x 44100 500 0.707)
print (spectral-centroid (magnitude-spectrum x) (spectrum-freqs 65536
```

```
44100))
```

The phase vocoder is one engine, `pvoc`, in C++ (a frame is sixty vector operations on 4096 bins, ten thousand times a minute): a fixed synthesis hop with the analysis hop derived from the stretch, zero-phase windowing, phases locked to the nearest spectral peak, pitch shift and formant move in the spectrum, envelope preservation through the cepstral envelope, denoising, three cross-synthesis modes and two phase effects, every parameter able to ramp over the file. `pvoc-stretch`, `pvoc-pitch`, `pvoc-pitch-formant`, `pvoc-formants`, `pvoc-cross`, `denoise`, `robotize` and `whisperize` are one-line wrappers around it.

```
(fft signal)
```

=> (list re im), each of length `next-pow2(length)`; the signal is zero-padded

```
(ifft (list re im)
```

) => real signal of the same length (a power of two)

```
(osc sr freqs table)
```

=> one sample per element of `freqs`, reading the wavetable with linear

```
(iir x b a)
```

=> y with $a_0 y[n] = \sum b[k] x[n-k] - \sum a[k>0] y[n-k]$; $a[0]$ need not be 1

```
(delay x d)
```

=> x delayed by d samples (fractional, linear interpolation), same length

```
(resample x factor)
```

=> x with its length multiplied by factor, by windowed-sinc interpolation

```
(autocorr x)
```

=> biased autocorrelation (sum / n) for lags 0 .. $n/2 - 1$; it decays with lag, which

```
(interleave (list a b ...)
```

) => one vector $a_0 b_0 a_1 b_1 \dots$

```
(deinterleave v channels)
```

=> (list a b ...): the channels of an interleaved vector

```
(local-maxima v)
```

=> indices k with $v[k] > v[k-1]$ and $v[k] > v[k+1]$ (interior points only)

```
(gather v indices)
```

=> the elements of v at the given (integer) indices, as a vector

```
(add-at! dst pos src)
```

=> dst, with src added in place starting at pos; dst must be long enough.

```
(comb x d g)
```

=> feedback comb $y[n] = x[n] + g y[n-d]$; (allpass x d g) => Schroeder allpass

(pvoc x opts)

after sparkle: fixed synthesis hop $I = \text{window/overlap}$, analysis hop $D = I/\text{stretch}$

(adsr sr gate a d s r)

=> an envelope following a gate signal (1 = on, 0 = off): attack a and

(lag x sr seconds)

=> x smoothed by a one-pole lowpass with the given time constant (parameter

(hpss x n hop kernel)

=> (list harmonic percussive): STFT with a periodic Hann window of n and

(ambi-gains order az el)

=> the $(\text{order}+1)^2$ encoding gains (SN3D, ACN) of a direction

(ambi-rotate B yaw)

=> B turned by yaw degrees about the vertical axis (positive to the left): within each order, the pair (m, -m) rotates by m yaw, exactly; B is a list of $(\text{order}+1)^2$ channel vectors

(hrtf-load path)

load a set from a CSV: each row azimuth, elevation, then the left ear's samples and the right's (as src/hrtf_kemar.csv, the MIT KEMAR compact set at 44100 Hz, found on the load path); => the number of directions. Once a set is loaded, hrtf takes the nearest measured direction (resampled to the requested rate); (hrtf-unload) goes back to the spherical-head model; (hrtf-loaded) => directions loaded, 0 for the model.

(hrir-model az el sr)

=> the spherical-head model, whatever is loaded (below)

(hrir az el sr)

=> (list left right): the nearest measured direction of the loaded set (the MIT KEMAR set is loaded from the load path on first use), resampled to sr; the spherical-head model when no set is available for each ear a head-shadow filter, whose high-frequency roll-off grows with the angle from that ear, and the interaural delay (Woodworth). No measured data: the model externalises and lateralises well enough to work with; real HRTFs can replace it (a WAV pair per direction convolved the same way).

(sine sr freq dur)

a sine of freq Hz lasting dur seconds

(noise n)

white noise in [-1, 1]; n may also be a signal, giving noise of its length (so an instrument can write (noise gate): offline as long as the gate, streamed forever)

(impulse n)

a 1 followed by n-1 zeros

(gen n amps)

one period of a wavetable with the given harmonic amplitudes, plus a guard point (n+1 samples, last = first), normalised to a peak of 1; for osc

(oscbank sr amps freqs table)

sum of oscillators; amps and freqs are lists of per-sample envelopes

(mix layers)

overlay (list (list position signal) ...) into one signal

(add-at dst pos src)

dst with src added starting at pos; dst grows if needed

(pan x pos)

a mono signal to (list left right), equal power; pos from -1 (left) to 1 (right)

(fade-in x n) (fade-out x n)

linear fades over n samples

(rms x)

root mean square

(normalize-peak x)

scale so the peak is 1; (normalize-rms x target) so the rms is target

(normalize-rms x target)

scale so the rms is target

(db x) (undb d)

amplitude to decibels and back

(next-pow2 n)

the smallest power of two $\geq n$ (fft sizes)

(window n a0 a1 a2)

generalised cosine window, periodic (so hann overlap-adds exactly at hop $n/4$); (hann n) (hamming n) (blackman n) are the usual ones

(hann n) (hamming n) (blackman n)

the usual windows, periodic

(hamming n)

the Hamming window

(blackman n)

the Blackman window

(car->pol spec)

(list re im) -> (list mag phase); (pol->car) the other way

(pol->car spec)

(list mag phase) -> (list re im)

(magnitudes spec) (phases spec)

from a complex spectrum

(spectrum-freqs n sr)

the frequency of each of the first $n/2$ bins

(magnitude-spectrum x)

the positive-frequency magnitudes of x, normalised so a full-scale sine reads 1

(complex-mul a b)

product of two (list re im) spectra

(conv x h)

linear convolution through the FFT, length $(+ (\text{length } x) (\text{length } h) - 1)$

(cepstrum mags)

real cepstrum of a full (symmetric) magnitude spectrum

(spectral-envelope mags order)

smooth envelope of a full magnitude spectrum: the "true envelope", a cepstral smoothing (first 'order' coefficients) iterated so that it rides on the harmonic peaks instead of averaging peaks and valleys. The order sets the resolution: about $sr / (2 f_0)$ of the sound (e.g. 150 for a voice at 44.1 kHz) follows the formants without rippling at the harmonics

(flatten-spectrum mags order)

the magnitudes divided by their envelope: the fine structure alone

(impose-envelope mags source order)

mags reshaped to carry the spectral envelope of source

(stft x n hop)

=> list of (list re im); (istft frames n hop) => signal by overlap-add

(istft frames n hop)

the signal back from stft frames by windowed overlap-add

(stft-magnitudes frames)

list of positive-frequency magnitude vectors, one per frame

(princarg x)

wrap a phase to $(-\pi, \pi]$

`(ramp v)`

a parameter given as a number or as `(list start end) -> (list start end)`

`(fftshift v)`

rotate a vector by half its length (zero-phase windowing)

`(pvoc-stretch x factor)`

time-stretch by factor with the pitch kept

`(pvoc-pitch x ratio)`

pitch-shift by ratio at the same length (formants move too)

`(pvoc-pitch-formant x ratio order)`

pitch-shift keeping the formants (order about sr/100: 400 at 44.1 kHz)

`(pvoc-formants x ratio order)`

move the formants by ratio, the pitch kept

`(pvoc-cross x y mode amount order)`

cross synthesis of x by y: mode 1 multiplicative, 2 spectral flattener (y's envelope on x), 3 morphing

`(robotize x)`

zero phases: a buzz at the frame rate

`(whisperize x)`

random phases: the spectral envelope on noise

`(denoise x threshold)`

magnitudes below threshold x the peak are zeroed (0.01 - 0.1)

`(gate-spectrum mags threshold)`

magnitudes below threshold times the frame's peak set to zero

`(spectral-morph a b t n hop)`

between two sounds of the same length: magnitudes interpolated linearly, phases taken from a below $t = 0.5$ and from b above

`(onset-strength x n hop)`

the spectral flux of every frame, as a vector (one value per hop)

`(onsets x sr n hop threshold)`

onset times in seconds, always starting with 0 (so the segments cover the sound from its beginning): then the peaks of the spectral flux (local maxima) above threshold $\times$ the flux's maximum, at least n samples apart from the previous onset, each placed at the sample where the energy jumps inside its frame (64-sample blocks: the block with the largest rise), so a hit is found where it starts, not a window early

`(onsets-adaptive x sr n hop threshold width)`

the same with a threshold that follows the material: a peak counts when it exceeds threshold $\times$ the moving median of the flux over width frames (odd, e.g. 9); for sounds whose loudness drifts, where a global level misses the quiet hits or admits the loud sustain

`(onsets-of-flux x sr n hop flux level)`

the shared picker: peaks of flux above level (a vector), spaced and refined

`(onset-refine x start n)`

the sample within $[start, start + n)$ where the energy rises most (64-sample blocks)

`(segments x sr times)`

the pieces of x between consecutive onset times (and the end), as a list

`(spectral-moment amps freqs order centroid)`

weighted moment of the frequencies about a centroid

`(spectral-centroid amps freqs)`

amplitude-weighted mean frequency

`(spectral-spread amps freqs)`

standard deviation around the centroid

`(spectral-skewness amps freqs)`

asymmetry of the spectrum about its centroid

`(spectral-kurtosis amps freqs)`

peakedness of the spectrum about its centroid

`(spectral-flux amps previous)`

rectified increase from the previous frame

`(spectral-irregularity amps)`

sum of absolute differences between neighbouring bins

`(spectral-decrease amps)`

weighted decrease relative to the first bin

`(spectral-flatness amps)`

geometric over arithmetic mean: 1 for noise, 0 for a pure tone

`(spectral-rolloff amps freqs fraction)`

frequency below which the fraction of the energy lies

(hfc amps)

high-frequency content

(zcr x)

zero-crossing rate per sample

(acf-f0 x sr)

fundamental by autocorrelation; 0 when no clear peak

(biquad type sr f0 q gain-db)

=> (list b a) of an RBJ biquad; type is one of "lowpass" "highpass" "bandpass" "notch" "peak" "lowshelf" "highshelf"

(apply-filter x coeffs)

run x through (list b a) as returned by biquad

(lowpass x sr f0 q) (highpass x sr f0 q) (bandpass x sr f0 q) (notch x sr f0 q)

one biquad, applied

(highpass x sr f0 q)

a highpass biquad, applied

(bandpass x sr f0 q)

a bandpass biquad, applied

(notch x sr f0 q)

a notch biquad, applied

(peak-eq x sr f0 q gain-db) (lowshelf x sr f0 q gain-db) (highshelf x sr f0 q gain-db)

the equaliser biquads, applied

(lowshelf x sr f0 q gain-db)

a low shelf, applied

(highshelf x sr f0 q gain-db)

a high shelf, applied

(dc-block x)

remove the DC offset; (dc-block-r x r) with pole radius r (default 0.995)

(dc-block-r x r)

a DC blocker with pole radius r

(reson x sr freq tau)

two-pole resonator at freq Hz with decay time tau seconds; the output lasts tau seconds, so a short excitation rings out

`(schroeder-reverb x sr rt60)`

four parallel combs into two allpasses; rt60 is the decay time in seconds (how long the tail takes to fall by 60 dB). The output is the input plus rt60 seconds, so the tail is not cut off.

`(resample-to x sr-in sr-out)`

`(bpf start segments)`

break-point function: piecewise linear segments as one vector; segments is a list of (list length end): (bpf 0 (list (list 4 1) (list 4 0))) rises then falls. Each segment's end value is excluded (it starts the next one).

`(envelope-follow x n)`

rms over hops of n samples, one value per hop

`(envelope-from-values v hop)`

piecewise-linear signal through successive values, hop samples apart

`(pan-azimuth x az)`

a mono signal to (list left right), equal power from its azimuth (-90 right .. 90 left; behind is folded onto the front)

`(speaker-ring n)`

n speakers evenly around, as azimuths starting in front and going left

`(pan-n x speakers az)`

a mono signal onto a horizontal ring of speakers (a list of azimuths) by amplitude panning between the two nearest speakers (VBAP in 2D); => a list of n channels

`(princarg-deg a)`

an angle wrapped to -180..180

`(ambi-encode x order az el)`

a mono signal at a direction as B-format: $(\text{order}+1)^2$ channels (ACN, SN3D)

`(ambi-add a b)`

two B-format signals of the same order summed

`(ambi-decode B speakers)`

B-format to a list of speakers, each (list az el) or a bare azimuth, by sampling the sound field at every speaker direction (the basic decoder), with max-rE weights per order for a smoother field off-centre

`(max-re-weights order)`

the per-channel max-rE weights of an order (Zotter & Frank): concentrates the energy vector

`(binaural x az el sr)`

a mono signal at a direction for headphones: convolved with the two ears' impulse responses of a spherical head (hrir); => (list left right)

`(channel sig k)`

one channel of a multichannel signal (a list of vectors); a vector is its own channel

`(stereo-add a b) (stereo-scale a g)`

stereo signals summed, scaled (written with channel, so they stream)

`(binaural-decode B sr)`

B-format for headphones: decoded to eight virtual speakers around the listener and two above, each rendered binaurally with the spherical head, summed; => (list left right). One expression, so it streams in a synth too.

`(normalize-peak-stereo s)`

a stereo (or any multichannel) signal scaled so its loudest sample is 1

`(ambi-render sources order)`

several (list x az el) placed and summed as one B-format signal

`(moving-source x order az-curve el-curve blocks)`

a signal whose direction changes: encoded block by block (blocks samples each) at the azimuth and elevation curves (vectors, one value per block)

`(nmf-separate x n hop k iterations)`

x split into k sources: NMF of its magnitude spectrogram (V = magnitudes, bins x frames) into k parts, then a Wiener mask per part (its share of $W H$ at every bin and frame) applied to the complex spectra and inverted; => (list (list source-1 ... source-k) $W H$). The sources add up to the input. Which part is which sound is for the caller to decide (their spectral shapes W and activations H say).

`(nmf-spectrogram frames n)`

the magnitude spectrogram of stft frames as a bins x frames matrix

`(nmf-sources x frames n hop W H groups)`

the sources of a factorization: groups is a list of lists of part indices, one source per group (a source may be several parts); Wiener masks, ISTFT

`(nmf-learn-parts examples n hop k iterations)`

supervised separation, step one: for each example sound (a list of vectors, one per source) learn k spectral parts by NMF; => W with k columns per source, and the groups (which columns belong to which source)

`(nmf-separate-with x n hop W groups iterations)`

supervised separation, step two: the mix x against known parts W (nmf-with-parts learns only the activations), one source per group

5.6 live

Real-time sound. The interpreter never runs on the audio thread: it posts commands (play this buffer at this time, stop that voice, set the gain) into a lock-free queue, and the audio callback drains the queue, advances a sample clock and mixes the voices into an N-channel bus. A voice plays a buffer (mono, or a list of channels) at its own sample rate, with amplitude, pan, rate and loop, from a start time on the clock, so sounds asked for slightly ahead land exactly where asked however late the program is. The device is miniaudio, with no window needed; a silent “null” device (the option (list (list "device" "null")), or the environment variable MUSIL_NULL_AUDIO) runs the same callback from a timer, which is how the test suite exercises everything but the ears.

```
load "live.mu"
audio-init                # 44100 Hz, 256 samples,
    stereo, started
var v (play-file "data/drums.wav")    # a voice
wait-for v
var lp (loop-buffer x sr)
set-rate lp 0.5
stop lp
var t (audio-time)
each (range 8) (function (k) (play-at (+ t (* k 0.25)) hit sr)) #
    sample-accurate, ahead of time
audio-quit
```

Synths. An instrument is an ordinary Musil function of its parameters whose body combines the *streamable* builtins: *osc*, *noise*, *adsr*, *lag*, the biquads (*lowpass* ... *highshelf*), *iir*, *dc-block*, *delay*, *comb*, *allpass*, *conv* (partitioned when streaming), *pan*, *list*, arithmetic and elementwise math (the list is (*streamable*)). Called normally it returns a buffer, because every one of those is a vector function in *signals*. Handed to (*synth* *f*) its body is compiled into a graph of nodes that run *the same loops* from *signals.h* one block at a time, and its parameters become hot: (*set-param* *id* 'cutoff 200 0.5) ramps the running sound; a parameter named *gate* is what *note-on*/*note-off* set. *synth-render* runs the graph offline with parameter curves, and the test suite checks it equals the direct call to float precision. Anything not streamable (*pvoc*, *resample*, ...) is refused by name; it is still a fine offline instrument.

```
function lead (gate freq cutoff) \
    (pan (* (adsr sr gate 0.01 0.2 0.5 0.4) (lowpass (osc sr freq
        saw-table) sr cutoff 2)) 0)
var a (synth lead)                # compiled, silent
set-params a (list (list 'freq 110) (list 'cutoff 1200))
note a 0.5                        # gate on, off after
    half a second (scheduled)
set-param a 'cutoff 300 1.5        # ramp while it plays
var y (lead gate-vector freq-vector 1200) # the same function,
    offline: a buffer
```

Loops. A loop is a function of the cycle number returning events, each (list *beat* *dur* *thunk*); (*live-loop* 'bass 4) calls the function *named* *bass* every four beats, slightly ahead of the clock (*lookahead*), and the thunks schedule sounds sample-accurately (*synth-ev*, *play-ev*, or your own *note-at*/*play-at*). Because the function is looked up by name at each cycle, redefining it while the loop runs is the live coding: the next cycle plays the new definition. *tempo*, *beat*, *stop-loop*; and the transformations of event lists, *fast*, *slow*, *shift*, *rev*, *every*, *degrade*, *cat*, *stack*, *steps*, are plain list functions. The scheduler runs in the interpreter's idle time (while the REPL or the IDE waits, and inside *sleep*), on one thread with everything else.

```
tempo 110
function drums (cycle) (stack (list
  (steps 1 (list 1 nil 1 nil) (function (x) (function (t d) (note-at
    t k 0.1))))))
  (every 4 cycle (function (e) (fast e 2)) (steps 0.5 (list nil 1) (
    function (x) (function (t d) (note-at t h 0.05)))))))
live-loop 'drums 4
function drums (cycle) ...           # redefine: next cycle it changes
```

Notes, patterns, a kit. `hz`, `note->midi`, `midi->hz`, `chord` for pitch names and chords; `pat` is a compact notation for steps, one string per cycle ("`bd ~ sn ~`": spaces are steps, `~` a rest, [`a b`] a subdivision, `a*n` a repeat), turned into events by `pat-events`, `drums` (a kit) and `melody` (a synth); `euclid`, `swing`, `humanize`, `sometimes`, `chance` shape rhythms; `poly` and `chord-ev` play chords on several instances. `kick`, `snare`, `clap`, `hat`, `ohat`, `acid`, `stab`, `pad` are ordinary instrument functions (`house-kit` makes the drums), so they render offline too; `tkick`, `rumble`, `that`, `tohat`, `tclap`, `perc`, `sub`, `hoover` (portamento through lag), `tstab`, `tpad`, `zap` and `techno-kit` are the harder set, with `duck` (a pad pumping under the kick), `sweep` and `accents`. A synth function may call your own helper functions (`sig` makes a constant as long as a signal, so (`osc sr (sig gate 190)table`) is a fixed pitch offline and streamed): the compiler inlines them. `examples/live_house.mu`, `live_techno.mu` and `live_trance.mu` are tracks meant to be edited while they play.

Controls and OSC. (`control 'cutoff 100 5000 800`) declares a named value with a range, (`toggle 'on 1`) an on/off one; `bind-control` makes synth parameters follow it, `set-control` and `control-value` read and write it from code, (`controls`) opens the window of sliders and check boxes and returns. `osc-listen` and `osc-map` let an incoming OSC message move a control; `osc-send` sends one; `osc-encode` and `osc-decode` are OSC in the language, over `udp-send` and `udp-receive`, for anything the conveniences do not cover. None of this is the evaluation port below: OSC carries numbers to controls over UDP, the port carries code over TCP.

Editors. (`serve 7770`) (in `system`; the IDE does it at start-up; the command line with `-serve 7770 -i`) opens a local port that evaluates the text it receives, in the session, as if typed. The VS Code extension in `editors/vscode/musil` sends the block around the cursor with `Cmd-Enter` (the file with `Cmd-Shift-Enter`) and gives `.mu` files syntax colouring, from a grammar generated with `help`. From a shell, `musil -send 7770 block.mu` (or with the code on `stdin`) sends and prints the reply, and any editor that can run a command can do the same; the two Musil processes stay separate sessions, the port only lets one hand code to the other.

(`audio-open sr block channels [opts]`)

open the audio device; if one is already open with the same sample rate and channels it is reused (voices cleared), otherwise it is closed and reopened. `opts`: (`list "device" "null"`) for the silent backend (tests, headless machines). Fails if a device is already open.

(`audio-close`)

stop and close the device

(`audio-start`) (`audio-stop`)

run or pause the callback; the clock advances only while running

(`audio-status`)

=> (`list (list "open" 0/1) (list "running" 0/1) (list "sr" sr) (list "block" n) (list "channels" n) (list "device" name) (list "time" seconds) (list "voices" n) (list "load" fraction) (list "peak" x))`)

(audio-devices)

=> list of the playback device names

(audio-time)

=> seconds on the engine's clock (sample-accurate, advances while running)

(play-buffer buffer sr amp pan rate loop at)

=> voice id. buffer is a vector (mono) or a list of channel vectors, at its own sample rate; at is the start time in seconds on the clock (0 = now)

(stop voice)

stop one voice; (stop-all) every voice

(master-gain g)

the output gain, applied to the whole bus

(voice-set voice "amp"|"pan"|"rate" value)

change a playing voice, ramped over a few blocks

(voices)

=> the ids of the voices playing or scheduled

(synth f)

=> id: compile the function f into a running instrument (silent until its parameters say otherwise). f's parameters become hot parameters; its body may use only the streamable builtins (see (streamable))

(set-param id name value [ramp-seconds])

change a synth's parameter, immediately or ramped

(synth-render f params seconds [sr])

=> the sound of f as the engine would stream it, computed offline block by block with its parameters set to params (a list of (list name value) or (list name buffer) for a value per sample); the same graph as (synth f), so the result equals f called on the same signals. No device needed.

(free id)

remove a synth from the graph

(synth-params id)

=> the parameter names of a synth, in the order of its function

(synths)

=> the ids of the running synths

(streamable)

=> the names of the builtins a synth function may use

(tempo bpm)

set the tempo; beats are counted from now

(beat)

=> the current beat (a number, fractional); (beat-time b) => the clock time of beat b; (bpm)
=> the tempo

(live-loop name beats)

start (or restart) the loop named name: the function of that name is called every cycle of 'beats'
beats; (stop-loop name) (stop-loops) (loops) => the names

(lookahead seconds)

how far ahead of the clock loops are scheduled (default 0.25)

(control name lo hi value)

declare a control (a range and a value); (toggle name value) an on/off one

(set-control name value) (control-value name) (controls-list)

=> (list (list name lo hi value toggle?) ...)

(bind-control name synth-id param)

a synth parameter follows the control (several bindings allowed); (unbind-control name)

(clear-controls)

forget every control

(osc-encode address args...)

=> the bytes of an OSC message as a string, for udp-send; (osc-decode bytes) => (list address args...) or nil: OSC in the language, on top of udp-send and udp-receive; osc-send, osc-listen and osc-map are conveniences over them

(osc-send host port address args...)

send an OSC message (numbers as floats, anything else as strings)

(osc-listen port)

receive OSC on a port (polled in the background); (osc-map "/address" control-name) the first number of a message at that address sets the control; (osc-stop)

(audio-init)

open the default device at 44100 Hz, 256 samples, stereo, and start it

(audio-init-with sr block channels)

the same with your numbers. With the environment variable MUSIL_NULL_AUDIO set (the test suite does), the silent null device is used. A device left open by a stopped program is reused (or reopened if the numbers differ), so running one live example after another just works.

(audio-quit)

stop everything and close the device

(audio-sr) (audio-channels) (audio-running?)

from audio-status

(audio-report)

print the status, one line per field

(play buffer sr)

play a buffer (a vector, or (list left right ...)) recorded at sr; => voice id

(play-with buffer sr opts)

the same with options: (list (list "amp" 0.5) (list "pan" -1) (list "rate" 2) (list "loop" 1) (list "at" 0.5)); at is a time in seconds on the audio clock

(play-file path)

play a WAV file (any channels) at its own sample rate; => voice id

(play-file-with path opts)

(play-at t buffer sr)

play at time t (seconds on the audio clock): sample-accurate however late the program is, as long as t is still ahead

(loop-buffer buffer sr)

play looping; stop it with (stop voice)

(set-amp voice a) (set-pan voice p) (set-rate voice r)

change a playing voice, ramped

(wait-for voice)

return when the voice has finished (polls the engine)

(wait-until t)

return when the audio clock has passed t seconds

(sequence steps)

play (list (list time buffer sr) ...) at their times, all scheduled at once

(note-on id) (note-off id)

set the synth's gate parameter to 1 or 0

(note id dur)

gate on now, off after dur seconds (sample-accurate, scheduled)

(note-at t id dur)

the same at time t on the clock

(set-params id pairs)

several parameters at once: (list (list 'freq 440) (list 'cutoff 800))

(play-synth f pairs)

compile f, set its parameters, gate it on; => id

(free-all)

free every synth

(synth-render-file f pairs seconds path)

render a synth offline through its graph and write a WAV

(ev beat dur thunk)

an event

(synth-ev beat dur id pairs)

a note on a synth: sets the parameters and gates it for dur

(play-ev beat buffer sr)

a buffer at a beat; (play-ev-with beat buffer sr opts) with play-with's options

(ev-beat e) (ev-dur e) (ev-thunk e)

(shift events beats)

move every event later by beats

(fast events factor)

squeeze the events in time by factor (2 = twice as fast); (slow events factor)

(rev events length)

play the cycle backwards (length = the loop's beats)

(every n cycle f events)

apply f to the events on every n-th cycle (cycle is the loop's cycle number)

(degrade events p)

keep each event with probability p

(cat lists length)

several cycles' worth of events one after another, each of length beats

(stack lists)

several event lists at once

(steps beats-per-step items thunk-of-item)

one event per item, evenly spaced; nil items are rests

`(midi->hz n) (hz->midi f)`

MIDI note numbers and Hz (69 = A4 = 440)

`(note->midi "C#4")`

a note name with octave (`#` or `b`) to a MIDI number; (`hz "A4"`) straight to Hz

`(chord root kind)`

the MIDI numbers of a chord: kind is 'maj' 'min' 'dom7' 'min7' 'maj7' 'sus4' 'dim'; root a note name

`(pat "bd ~ sn ~" beats)`

one string per cycle: tokens separated by spaces are steps of equal length, `~` is a rest, `[a b]` subdivides a step, `a*n` repeats a token `n` times; $\Rightarrow$ `(list (list beat dur token) ...)` spread over 'beats'. It is a notation for steps, nothing more: `(pat "x ~ x ~" 4)` equals `(steps 1 (list "x" nil "x" nil) ...)`

`(pat-events pattern maker)`

events from a pattern: maker gets the token and returns a thunk (`t d`)

`(euclid hits steps)`

the Euclidean rhythm as a list of 1 / nil, for steps: `(euclid 3 8) = x ~ ~ x ~ ~ x ~`

`(swing events amount)`

push the off-beat eighths later by amount (0 none, 0.33 heavy), in beats

`(humanize events beats)`

move every event by a random amount up to +/- beats

`(sometimes p f events)`

apply `f` to the events with probability `p`, per cycle

`(chance p events)`

keep each event with probability `p` (an alias of `degrade` with the argument order of `sometimes`)

`(poly f n)`

`n` instances of the synth function `f`; $\Rightarrow$ `(list id ...)`

`(chord-ev beat dur ids midis pairs)`

a chord: each instance gets a pitch (as freq) and the pairs, and is gated for `dur`

`(sig gate v)`

the constant `v` as a signal as long as gate: `(osc sr (sig gate 190) table)` is a fixed 190 Hz both offline (a vector) and streamed (a node); plain 190 would be one sample offline

`(kick gate)`

a sine with a pitch drop and a click, saturated

(snare gate)

bright noise and a short 190 Hz tone

(clap gate)

band-limited noise with a spread attack

(hat gate) (ohat gate)

closed and open hi-hats: highpassed noise, short and long

(acid gate freq cutoff res)

a sawtooth into a resonant lowpass whose cutoff follows an envelope, then drive (a moving cutoff is a streaming feature: use synth-render for a buffer of it)

(stab gate freq cutoff)

the chord voice: two detuned saws and a sub octave, lowpassed

(pad gate freq cutoff)

slow detuned saws for the breakdown

(house-kit)

synths for kick snare clap hat ohat as (list (list 'kick id ...) (kit-get kit 'name))

(drums kit pattern-string beats)

events for a kit from a pattern whose tokens are bd sn cp hh oh

(melody id pattern-string beats pairs)

notes for a synth from a pattern of note names (a2 c3 ...) and rests

(tkick gate)

a hard kick: deep pitch drop, click transient, saturated

(rumble gate)

the kick's tail: a low sine with a long decay through a comb, lowpassed

(that gate) (tohat gate)

metallic hats: ring-modulated squares through a highpass

(tclap gate)

a wide clap: noise through a resonant bandpass with two attacks

(perc gate freq)

a tuned percussion: noise burst plus a pitch-dropping tone

(sub gate freq)

a sub bass: a sine an octave under freq, with a hint of the fundamental

(hoover gate freq cutoff glide)

detuned saws with portamento (the frequency is lagged by glide seconds), a dark lowpass

(tstab gate freq cutoff)

a dark stab: squares through a lowpass with a fast decay

(tpad gate freq cutoff amp)

a pad with an amp parameter, for ducking (duck)

(zap gate freq)

a fast downward sweep: saw with a pitch envelope, for accents

(techno-kit)

synths for kick rumble hat ohat clap perc, as (list (list 'kick id) ...); tokens for drums: bd rm hh oh cp pc

(drums kit s beats)

also understands rm (rumble) and pc (perc) tokens

(duck ids beats depth)

ducking events on the beats: every beat the amp parameter of each id drops to depth and comes back in a quarter beat (the pad's "pumping" under the kick)

(sweep id name from to beats)

ramp a parameter from one value to another over beats (returns at once)

(accents events amount)

push every event a little later except the ones on the beat (a lazy feel)

(supersaw gate freq cutoff spread)

seven detuned saws (spread in cents, ~12) through a lowpass: the lead and pad of trance

(pluck gate freq cutoff)

a short bright saw pluck: the arpeggio voice (its cutoff falls with the note)

(offbass gate freq)

the off-beat bass: a saw and a sub, tight envelope, lowpassed

(riser gate rate)

a build: noise whose highpass opens with an envelope, over rate seconds (streaming: a moving cutoff)

(trance-kit)

the drums as synths: kick clap hat ohat (house sounds, a harder kick), tokens bd cp hh oh

(arp midis mode steps beats)

an arpeggio as a list of MIDI notes over steps: mode 'up 'down 'updown 'random

(arp-events id midis mode n beats pairs)

the arpeggio of n steps over beats, played on a synth

(gater ids pattern-string beats depth)

the trance gate: the amp parameter of the ids follows a pattern of x and ~ (x = open, ~ = closed to depth), for pads with an amp parameter (tpad)

(roll id beats n)

a snare or clap roll accelerating over beats: n hits spaced tighter and tighter

5.7 music

The score: things placed in time. A score is a record (name, sample rate, events, layout); an event has an onset, a duration, a kind, a source, a gain and a position (azimuth and elevation, for the spatial layouts). What an event holds is decided by what is placed:

```
var s (score "study" 44100)
event s 0 3.5 "bells.wav" # a sound
    file, read when rendered, cached
event s 2 3.6 buffer # a vector,
    or a list of channel vectors
event s 4 1 (note db 'Vn 'C5 'mf 'ord) # a sound
    from a database (nearest pitch, shifted)
event s 5 2 (instrument pluck (list (list 'freq 440))) # a synth:
    an instrument function and its parameters
event s 6 1 (call my-function (list 1 2 'dur)) # any
    function, called when rendered ('dur = the duration)
event-at s 7 1 my-function 90 0 # a bare
    function is called with the duration; placed at the left
render s "study.wav" "stereo" # layouts:
    "mono" "stereo" "binaural" "ambi N", a ring, or N channels
play-score s 0.8 # through
    live, ahead of the clock, scaled
display s # the
    player: the roll, playable, editable
```

A sound longer than its event is faded out at the end; a shorter one is left as it is; a synth's gate is the duration and its release plays on. A mono sound is spatialised at its position; a sound with as many channels as the layout goes in as it is. **play-score** (with a gain for the whole; **play-score-from** from a time) renders every sound first, then schedules everything ahead of the clock; it refuses an instrument that does not stream, naming the event; **play** and **play-file** of **live** still play a buffer or a file.

The player (**display**) is the roll as a window: one row per instrument for the notes, in orchestral order, then a row per kind (files, buffers, synths, calls), overlapping events in lanes; A D and the wheel pan and zoom in time, W S and Z X scroll and zoom the rows, R re-sets; hovering shows an event's data and the time; a click in the background places the cursor; **Play** (or Space) plays from the cursor and the cursor follows the clock (**Stop** while it plays); dragging a bar moves its event, dragging its end changes its length, and the score's own numbers change, so the next Play, Render or Export has the edit; **Render...** asks for a file name and writes the stereo mix; **Export...** asks for a name and writes a Musil file defining (**generated-score**), a function that rebuilds the score (notes, files, synths and calls by name; buffers are skipped). The window is on the FLTK thread and hands every action to the interpreter as a line of Musil, so it works the same from the command line and from the IDE.

`score-shift!`, `score-scale!`, `score-select`, `score-sort!`, `score-remove!`, `score-merge` are the transformations; `event-move!`, `event-length!`, `event-gain!`, `event-place!` edit one event.

Databases. A database in the *SOL layout (TinySOL, OrchideaSOL, FullSOL) is a feature file next to a folder of the same name holding the sounds by family and instrument (TinySOL.spectrum.db and TinySOL/); the file's first line gives the type, block, hop and number of coefficients, then one line per sound: its path relative to the sounds' folder and its features. The sounds' folder is found by content, whatever its name (TinySOL/, tinysol/, a copy called otherwise): the folder next to the feature file that holds its sounds; and each sound is found by its file name anywhere under that folder, so a copy whose folders are named otherwise than the feature file says (Strings/Vn/ for Strings/Violin/) or spells sharps as _ loads too. `db-load` reads the metadata (the file names carry instrument, technique, pitch, dynamics) and the features, in C++, and never the sounds: those are opened when a note is rendered or played, and cached. `db-query` (nil matches anything, a list matches any of its members), `db-grep` (a regular expression on the file name), `db-find` (a substring), `db-between` (a pitch range), `db-nearest` (the closest features), `db-instruments` (in orchestral order), `db-techniques`, `db-dynamics`, `db-pitches`, `db-range`, `db-features`, `db-path`, `db-available?`, `db-available`, `db-instruments-available`. `db-gen` makes a feature file from a folder of sounds (Orchidea's dbgen, in C++: the energy-weighted average spectrum at 44100 Hz, as `spectrum`, `logspec`, `specpeaks`, `specenv`, `mfcc` or `moments`; the bundled TinySOL features are reproduced exactly); `db-make` generates and loads. Several databases whose features agree load as one: (`db-load (list "a.db" "b.db")`) or `db-merge`; every entry keeps its own sounds' folder, and `note` draws on all of them. `note` finds the entry for an instrument, pitch, dynamics and technique, or the nearest available pitch of that instrument, and shifts it by resampling; only sounds present on disk are candidates, so a feature file describing more than is present still works. A sharp in a pitch is written as a string ("D#5"): a # in a symbol would start a comment. The bundled `examples/data/microsol` (MicroSOL: oboe, horn, violin, cello, C4–G4, 32 sounds) makes every example run from a fresh clone; the full TinySOL is fetched into `datasets/` (ignored by git) with `./fetch_tinysol.sh`, and each example has a commented line to switch to it.

```
(pitch->midi "D#5")
```

=> 75: a note name with sharps (#, _) or flats (b) and an octave (C4 = 60); -1 if not a pitch

```
(midi->pitch 75)
```

=> "D#5"

```
(db-read path)
```

=> (list type block hop ncoeff entries): the feature file parsed; each entry a record (file instr tech pitch dyn other midi features), the file relative to the database's root

```
(db-gen folder path type block hop ncoeff)
```

=> the number of sounds analysed: every WAV under folder (recursively), its features written to path in the database format (first line: type block hop ncoeff; then the file's path relative to folder and the features); the file names carry the metadata (instr-tech-pitch-dyn-other). Sounds that cannot be read are skipped with a message.

```
(db-locate root file)
```

=> the path on disk of a sound named file (a path from a feature file) under root, found by its file name whatever the folders are called; "" when it is not there. (db-index-size root) => how many sounds root holds

```
(render s "out.wav" "stereo")
```

writes the mix; layouts: "mono" "stereo" "binaural" "ambi N", a list of speaker azimuths (a ring), or a number of channels (multichannel events as they are)

(play-score s gain)

plays it through live, ahead of the clock (synths compiled, the rest rendered), everything scaled by gain

(display s)

the roll: rows by kind (notes in orchestral order, then files, buffers, synths, calls); WASD and + - 0 navigate in time, hovering shows an event and the time

(score name sr)

a new, empty score at a sample rate

(score-events s) (score-sr s) (score-duration s)

its events (in the order they were added), rate, end time

(event-end e)

when the event ends

(event s at dur what)

add an event; what is a path, a buffer, a note, an instrument, a call, or a function; => the event record (its at and dur can be changed later with put!)

(event-at s at dur what az el)

the same, placed at an azimuth and an elevation (degrees, 0 in front, left positive)

(payload->event what)

the event record of a payload (kind and source), without its time

(filename path)

the last part of a path, for labels

(event-gain! e g)

the gain of an event (1 by default); (event-move! e at) (event-length! e dur) (event-place! e az el)

(instrument f params)

a synth event: f a streamable instrument function (gate first), params its parameters as (list (list 'freq 440) ...); the gate is the event's duration

(call f args)

a call event: f applied to args when the score is rendered; its result (a vector or a list of channels) is the sound. A function taking the duration can ask for it: pass 'dur among the args, it is replaced by the event's duration in seconds

(fn-name f)

the name a function prints as, for labels

(load-sound path)

a WAV as (list sr channels), from the cache

(clear-sound-cache)

forget the loaded files

(to-rate channels from to)

channels resampled from one rate to another (nothing to do when equal)

(render-event e sr)

the sound of an event at the score's rate: a list of channels, cut to its duration with a fade at the end when the sound is longer, gain applied

(fit-duration x dur sr)

x cut to dur seconds with a fade-out over the last 20 ms (or a quarter of dur), if longer

(layout-channels layout)

how many channels a layout has

(place chans layout az el sr)

an event's channels into the layout's channels: a mono sound is spatialised at its direction; a sound with as many channels as the layout goes in as it is; anything else is summed to mono first

(score-render s layout)

the whole score as channels at its rate

(render s path layout)

the score written as a WAV; => the channels

(play-score s gain)

play the score now (the device is opened if needed) and wait until it ends; files, buffers, notes and calls are rendered first, then everything is scheduled ahead of the clock; synths are compiled and gated on time (an instrument that is not streamable is refused, naming the event); gain scales everything (1 as rendered)

(play-score-from s gain from)

the same from a time in seconds: events already over are skipped, one under way starts in the middle

(score-schedule s gain from)

schedule the score's events on the clock and return at once: => (list synth-ids end-clock-time start-clock-time); free the synths when it is over

(score-export s path)

the score as a Musil file defining (generated-score), a function that rebuilds it: notes, files, synths and calls are written by name (buffers cannot be, and are skipped with a note); load the file and call (generated-score) to get the score back

(db-key db)

what identifies a database for export: its file, or the list of files of a merged one

(to-source v)

a value as Musil source text: numbers, strings, symbols and lists of them

(instrument-rank instr)

where an instrument code sits in the orchestral order (unknown ones after)

(score-rows s)

the rows of the roll: one per instrument (orchestral order) for the notes, then one per kind: files, buffers, synths, calls; => (list (list label events) ...)

(score-roll s)

the roll as a figure: rows and bars (each bar keeps the event's own at and dur values, so the player's edits reach the score)

(event-lanes events)

a lane number per event such that events sharing a lane do not overlap in time

(display s)

open the score in the player: the roll, playable, editable, renderable (see player.h); without the player (no FLTK) the roll is shown as a figure

(db-load path)

load a database from its feature file: TinySOL.spectrum.db with the sounds in TinySOL/ next to it (the paths in the file are relative to that folder); when no such folder exists the sounds are looked for next to the file itself. => a record with 'path, 'root (the sounds' folder), 'type, 'block, 'hop, 'ncoeff and 'entries (records: file instr tech pitch dyn other midi features). The sounds are not read: notes open them when rendered or played.

(db-sounds-folder dir entries)

where the sounds of a feature file in dir are: the folder next to it (or dir itself) under which the first entry's file exists, whatever the folder is called (TinySOL/, tinysol/, a copy named otherwise); dir when nothing is found (a note then says the sound is missing)

(db-merge dbs)

one database out of a list of several, when their features agree (same type and number of coefficients): the entries of all, each remembering its own sounds' folder; a note then draws on every set at once. (db-load (list "a.db" "b.db")) is the same.

(db-size db) (db-instruments db) (db-techniques db) (db-dynamics db) (db-pitches db)

what the database holds

(db-query db instr pitch dyn tech)

the entries matching the given values; nil matches anything, a list matches any of its members: (db-query db 'Vn nil 'mf nil) every mf violin sound, (db-query db (list 'Vn 'Va) nil (list 'pp 'p) 'ord) quiet upper strings

(db-grep db pattern)

the entries whose file name matches a regular expression: (db-grep db "Fl-ord-C.-pp")

(db-find db text)

the entries whose file name contains the text: (db-find db "Vn-ord-C5")

(db-between db instr lo hi)

the entries of an instrument between two pitches (names or MIDI numbers), inclusive

(db-nearest db entry n)

the n entries whose features are closest (euclidean) to an entry's: sounds alike

(db-available db)

the entries whose sounds are on disk; (db-instruments-available db) their instruments

(db-range db instr)

the lowest and highest MIDI note of an instrument, as (list lo hi)

(db-path db entry)

the sound file of an entry, on disk: under the sounds' folder at the path the feature file gives when it exists, else found by its file name anywhere under that folder (the folders of a copy may be named otherwise, and a sharp written # in the file may be _ on disk); the path as the file gives it when the sound is not there at all

(db-available? db entry)

is the sound file of an entry on disk? (a feature file may describe more sounds than are present)

(db-features entry)

the feature vector of an entry

(db-make folder path type block hop ncoeff)

analyse every WAV under a folder into a feature file (db-gen) and load it

(note db instr pitch dyn tech)

a note event: the entry matching instrument, pitch, dynamics and technique, or the nearest available pitch of that instrument (same dynamics and technique when they exist), shifted by resampling to the pitch asked for. Only entries whose sound is on disk are candidates.

(note-path e)

the sound file of a note event, on disk

(note-duration e)

how long the note's sound is, in seconds (after the shift)

(score-shift! s dt) (score-scale! s f)

every event moved by dt seconds; every onset and duration multiplied by f

(score-select s f)

the events for which `f` holds

```
(score-sort! s)
```

the events ordered by onset

```
(score-remove! s e)
```

an event taken out

```
(score-merge s1 s2)
```

a new score with the events of both (`s2`'s after `s1`'s end when shifted with `score-shift!`)

```
(score-print s)
```

the events, one line each

5.8 plot

A figure is `(list title layers options)`; `plot.mu` builds it and `show` opens it in a window of its own and returns, from the command line and from the IDE alike; the interpreter keeps the windows alive while it waits or sleeps, so several plots can be open while a program goes on, and a script that ends with windows open stays until they are closed. `save-png` writes a file. `subplots` puts several figures in one grid. Keys in a window: +/- zoom, W A S D pan, arrows orbit a surface, 0 or `r` reset, `e` exports a PNG under an automatic name, Esc or `q` closes; the mouse drags to pan or orbit, the wheel zooms; the **Export...** button asks for a file name and writes the figure as shown. Layers are lines, points, bars, an image (a matrix as a heat map) or a surface (a matrix in 3D). The one-call helpers build and show in one go; the ready-made figures know about signals and data.

```
load "plot.mu"
var fig (figure "two sines")
add-line fig nil (sin (/ (range 100) 8)) "sin"
add-line fig nil (cos (/ (range 100) 8)) "cos"
set-labels fig "sample" "value"
show fig
save-png (spectrogram x sr 1024 256) "/tmp/spec.png"
show (scatter-clusters petals (kmeans-labels km) 3)
```

```
(save-png fig path [w h])
```

render a figure to a PNG file (900 x 560 by default)

```
(show fig [w h])
```

open the figure in a window of its own and return; the program goes on and the window stays (the idle hook keeps it alive); several can be open. Skipped when `MUSIL_NOSHOW` is set.

```
(plot-windows)
```

=> how many plot windows are open; `(close-plots)` closes them

```
(figure title)
```

an empty figure

```
(fig-title fig) (fig-layers fig) (fig-options fig)
```

the three parts of a figure

`(fig-layers fig)`

the layers

`(fig-options fig)`

the options

`(add-line fig x y label)`

x may be nil to use 0, 1, 2, ...

`(add-scatter fig x y label)`

points; `(add-bars fig x y label)` bars

`(add-bars fig x y label)`

bars

`(add-image fig M)`

a matrix as a heat map, row 0 at the bottom

`(add-surface fig M)`

a matrix as a 3D surface (a figure with a surface shows only that)

`(add-layer fig layer)`

add a raw layer (list kind data ...)

`(x-or-index x y)`

x, or 0 1 2 ... when x is nil

`(set-option fig key value)`

keys: "xlabel" "ylabel" "xmin" "xmax" "ymin" "ymax"

`(set-labels fig xlabel ylabel)`

axis labels; `(set-xrange fig lo hi)` `(set-yrangle fig lo hi)` axis limits

`(set-xrange fig lo hi)`

x-axis limits

`(set-yrangle fig lo hi)`

y-axis limits

`(subplots title figs rows cols)`

a grid of figures, drawn together; rows or cols may be 0 (auto)

`(add-subplot fig sub)`

add one more figure to a grid

`(plot y)`

or `(plot-xy x y)` a line

`(plot-xy x y)`

a line of `y` against `x`

`(plot-lines ys labels)`

several lines sharing the index axis

`(scatter x y)` `(bars y)` `(image M)` `(surface M)`

build and show a one-layer figure

`(bars y)`

build and show a bar chart

`(image M)`

build and show a heat map

`(surface M)`

build and show a 3D surface

`(waveform x sr)`

a signal against time in seconds

`(spectrum x sr)`

magnitude spectrum in dB against frequency

`(spectrogram x sr n hop)`

STFT magnitudes in dB as an image: time along `x`, frequency along `y`

`(scatter-clusters X labels k)`

the rows of an `n x 2` matrix coloured by cluster

`(histogram-figure v bins)`

a bar chart of `(histogram v bins)`

6 Extending Musil from C++

The interpreter is one header. A host includes it, registers functions, and runs code:

```
#include "musil.h"
musil::Interp I;
musil::make_env(I);                                     // the C++ halves of the
    libraries
I.def("hello", [] (musil::vlist& a, musil::Interp& i) -> musil::vp_ptr {
    return musil::v_str("hello, " + i.str(a[0]));
```

```
}, 1, 1); // name, function, min and
           max arguments
I.run("print (hello \"world\")");
```

A builtin receives its evaluated arguments as a `vlist` and the interpreter. `eval` has already checked the argument count. Inside, `i.num(v)`, `i.scalar(v)`, `i.index(v)`, `i.str(v)`, `i.list(v)` and `i.fn(v)` return the payload or raise a Musil error naming the builtin; `i.bad("message")` raises one with your text. Values are made with `v_num`, `v_arr`, `v_str`, `v_list`; `i.call_fn(f, args)` calls a Musil function; `i.yield_fn` is called every 1024 evaluations for hosts that service an event loop.

Document a builtin with a comment above it in the form `// (name args) description`; `tools/gendoc.py` turns those comments into `help.txt` and into this manual. A library is complete when it has `name.h`, `name.mu`, `tests/test_name.mu` and `examples/reference_name.mu`.

7 Examples

All in `examples/`, all run by `ctest`. Run them from that directory.

```
reference*.mu
hello, if, while, procs, math, strings, arrays, files
music_demo.mu
chords.mu
matrices.mu, iris.mu
stft, filters, reverb, cross_synth, additive, features
phase_vocoder.mu
plots.mu, iris_plot.mu
live_play.mu, live_chorale.mu, live_probe.mu
live_synth.mu, live_offline.mu
live_loops.mu, live_controls.mu, live_session.mu
live_house.mu, live techno.mu, live_trance.mu
onsets.mu, transients.mu
spatial.mu, flyby.mu, flyby_controls.mu, live_spatial.mu
nmf.mu, make_audio_demos.mu
simple_score.mu, loaddb.mu, random_music.mu, random_collage.mu
bootstrap.mu
```

one runnable tour per library
the language, one topic each
temperament, scales, pitch-class sets,
pitch-class sets as vectors: distances,
linear algebra; PCA, k-means and KNN
signal processing on recordings
time stretching, pitch shifting with ar
waveforms, spectrograms, responses,
sounds through the device; a four-voi
instruments as functions, played and
patterns redefined while they run; cor
tracks from the kits, the step notation
onset detection by spectral flux; harm
sounds placed and moved in 3D: bina
source separation of a four-stem mix
a score written by hand (a Bach chor
Musil evaluating Musil: a metacircula